操作系统

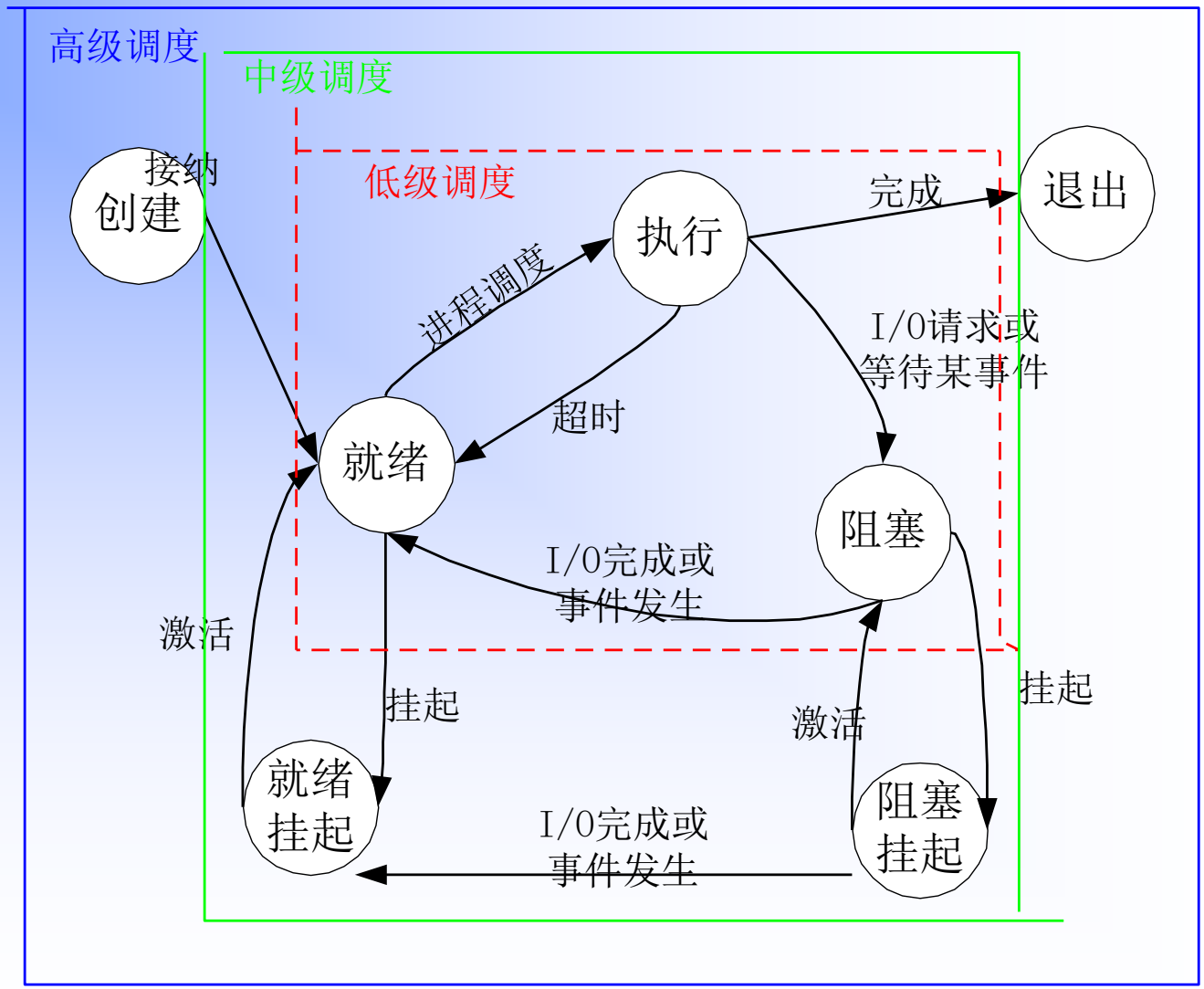
调度与死锁

邓一星 2025.3.25

4.1调度类型与准则

调度类型

- 高级调度
- 低级调度
- 中级调度



调度的层次

高级调度

又称作业调度、宏观调度

- 任务：决定将外存上后备队列中的哪些作业调入内存。
- 调度工作决定
  - 接纳多少作业：取决于多道的程度，即内存允许放多少个作业。
  - 接纳哪些作业：有调度算法决定。
- 适用于批处理系统

低级调度

又称进程调度、微观调度

- 任务：决定就绪队列中的哪些进程将获得处理机。
- 调度方式
  - 非剥夺式
  - 剥夺式
- 抢占原则
  - 时间片
  - 优先权
  - 进程长短
- 适用于分时、实时、批处理系统

中级调度

又称对换程序

- 主要作用：内存和外存对换区之间进行进程对换，以解决内存紧张问题。

进程调度方式

- **不可剥夺方式**

  不允许其它进程强占正在运行进程占有的处理机。

- **可剥夺方式**

  允许一个进程按照某种原则，抢占其它进程占有的处理机。

进程调度时机

- 进程退出
- 进程阻塞
- 新进程创建
- 中断发生
- 时钟中断

调度的性能准则

- 面向用户的准则
  - 响应时间快

    响应时间：从用户通过键盘提交请求到首次得到响应的时间
  - 周转时间短：

    周转时间：作业从提交到完成的时间间隔。
  - 优先权准则
  - 截止时间的保证
-

调度的性能准则

- 面向系统的准则
  - 系统吞吐量

    单位时间内完成的作业数。
  - 处理机利用率
  - 各类资源平衡利用
  - 公平

4.2 调度算法

- 先来先服务调度算法
- 短作业（进程）优先调度算法
- 时间片轮转调度算法
- 优先权调度算法
- 多级反馈队列

先来先服务FCFS与短作业优先SJF

FCFS

- 算法思想
  - 对于作业调度，从后备作业中选择最先进入该队列的作业，将他们调入内存，为它们分配资源、创建进程，然后放入就绪队列。
  - 对于进程调度，从就绪队列中选择最先进入该队列的进程，分配处理机，使之运行。
- 特点
  - 易于实现
  - 有利用长作业，短作业不满。

SJF

- 算法思想
  - 短作业优先是从后备队列中选择估计运行时间最短的作业，将它们调入内存。
  - 短进程优先是从就绪队列中选择估计运行时间最短的进程，将处理机分配给它，使之执行并一直到完成或因发生某事件而阻塞放弃处理机时，再重新调度。
- 特点
  - 极端情况下，长作业得不到调度。
  - 作业或进程的长短只能估计，不准确。
  - 完全不考虑紧迫程度，使紧急事件得不到处理。

FCFS与SJF比较

作业情况	进程名	A	B	C	D	E	平均
	到达时间	0	1	2	3	4	
调度算法	服务时间	4	3	5	2	4	
	完成时间	4	7	12	14	18	
FCFS	周转时间	4	6	10	11	14	9
	带权周转时间	1	2	2	5.5	3.5	2.8
	完成时间	4	9	18	6	13	
SJF	周转时间	4	8	16	3	9	8
	带权周转时间	1	2.67	3.2	1.5	2.25	2.1

SRTN--最短剩余时间优先算法

- 算法思想
  - 当新任务到达时，系统比较新任务的总执行时间与当前任务剩余执行时间，若新任务更短则立即抢占处理机。
- 优点
  - 进一步减少平均等待时间
- 缺点
  - 长任务可能因持续被短任务抢占而“饥饿”

HRRN--高响应比优先算法

- 算法思想
  - 通过动态计算任务的响应比 （(等待时间 + 要求服务时间) / 要求服务时间）来决定调度顺序
- 优点
  - 避免长作业“饥饿”，提升系统公平性
- 缺点
  - 每次调度需计算所有任务的响应比，计算开销较大

示例

进程	到达时间	执行时间（分钟）
P1	0	6
P2	2	3
P3	4	2
P4	5	4

时间片轮转调度算法

- 算法思想
  - 进程按FCFS在就绪队列排队，调度程序把CPU分配给队首进程，令其执行一个时间片，一个时间片执行完毕将进程排在队尾。
- 时间片大小的确定
  - 响应时间T
  - 就绪队列的进程数N
  - 系统的处理能力

系统处理能力比较

时间片q=1和q=4时进程的平均周转时间和平均带权周转时间

	进程名	A	B	C	D	E	平均
	到达时间	0	2	4	6	8	
	服务时间	3	6	4	5	2	
	完成时间	4	18	17	20	15	
q=1	周转时间	4	16	13	14	7	10.8
	带权周转时间	1.33	2.67	3.25	2.8	3.5	2.71
	完成时间	3	17	11	20	19	
q=4	周转时间	3	15	7	14	11	10
	带权周转时间	1	2.5	1.75	2.8	5.5	2.71

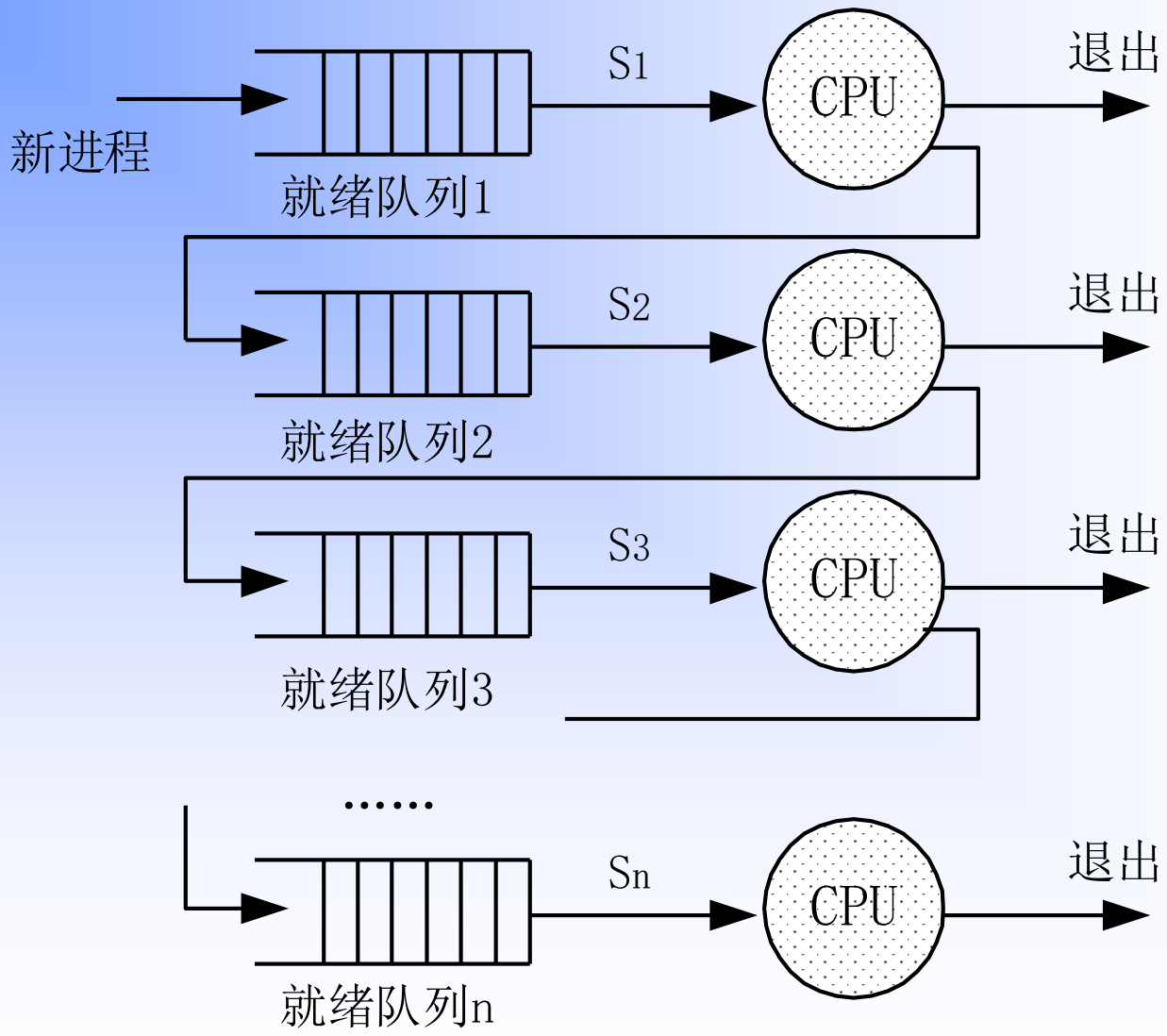
优先权调度算法

- **算法思想**
  - 从后备队列中选择若干优先权最高的作业，将它们调入内存。
  - 或从就绪队列中选择优先权最高的进程，将处理机分配给它。
- **优先权类型**
  - 静态优先权
    - 确定因素：进程类型、进程对资源的需求、用户要求。
  - 动态优先权
    - 确定因素：等待时间、运行时间。
- **特点：综合考虑各种情况**

多级反馈队列调度算法

- 算法思想
  - 根据作业的性质和类型不同，将就绪队列再分为若干个子队列，每个进程分属于一个队列。
  - 在多级队列的基础上，不但设多个队列，且为每个队列赋予不同的优先权，第一个队列的优先权最高，第二个队列次之，其余队列的优先权逐个降低。
  - 各个队列中的进程执行时间片大小逐渐增大。
  - 新进程投入第一个队列。
  - 调度从第一个队列进行，仅当第一个队列为空时，才调度第二个队列中的进程。

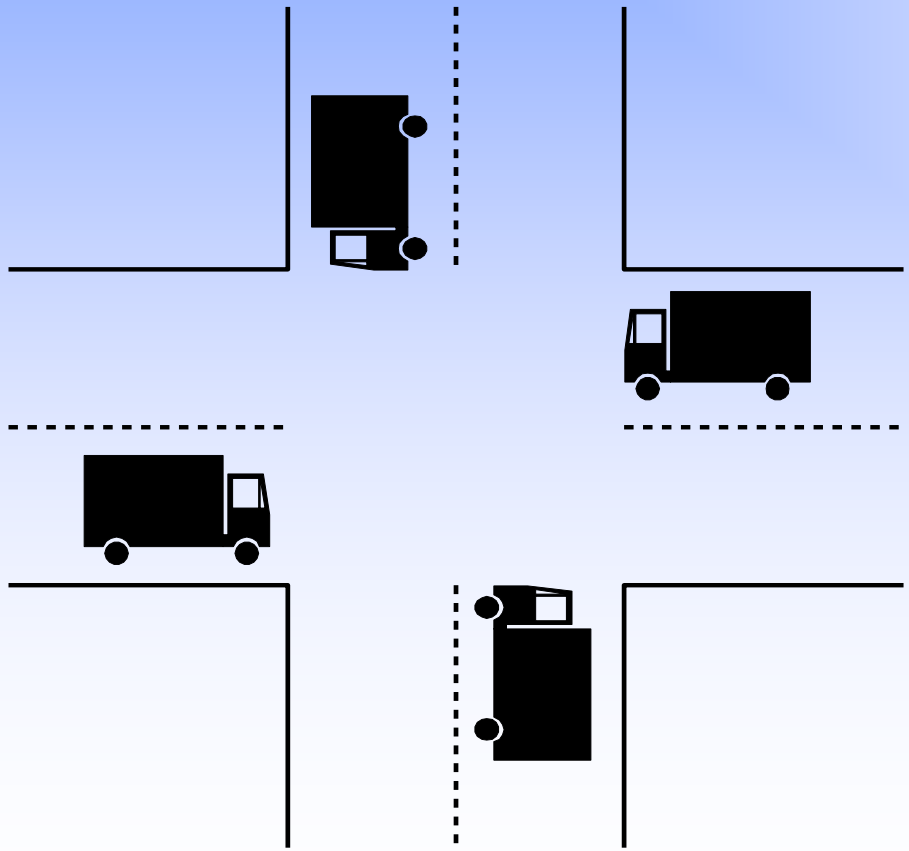
多级反馈队列调度算法



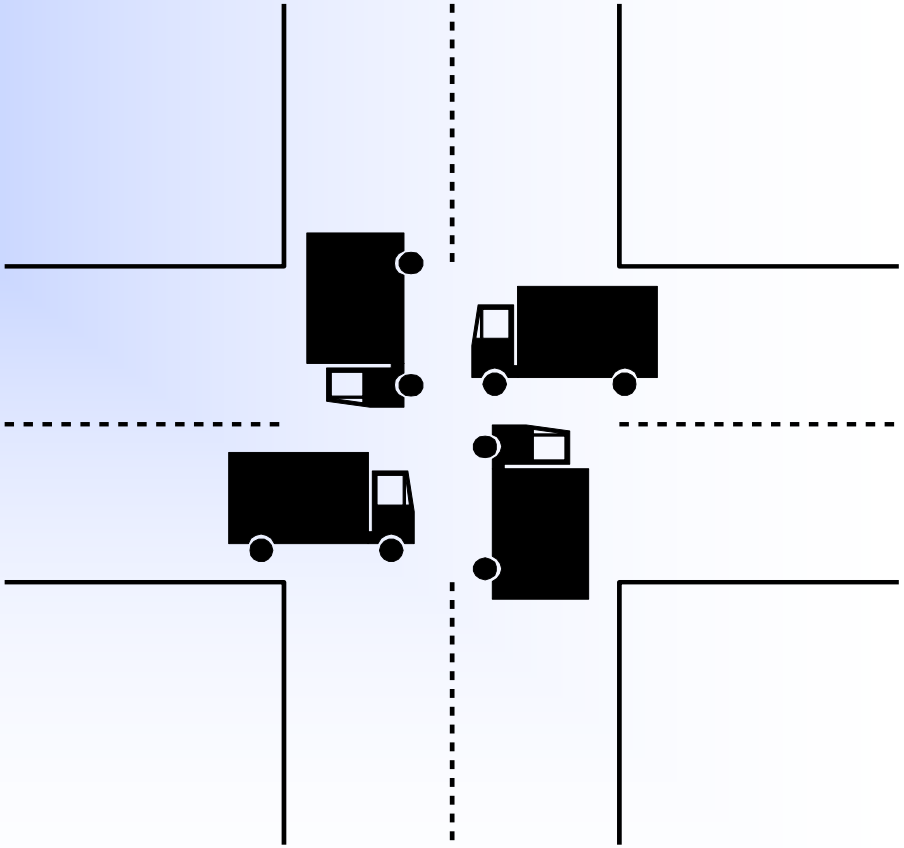
时间片 S1 < S2 < S3< … < Sn

4.3 死锁的基本概念

一组竞争系统资源或相互通信的进程相互的“永久”阻塞。若无外力作用，这组进程将永远不能继续执行。



(a) 可能死锁



(b) 死锁

产生死锁的原因

A、B分别代表某种资源

进程P

……
get(A);
……
get(B);
……
release(A);
……
release(B);
……

进程Q

……
get(B);
……
get(A);
……
release(B);
……
release(A);
……

- 资源数 < 要求该种资源的进程数
- 进程的推进顺序非法

产生死锁的四个必要条件

互斥条件

请求保持条件

不剥夺条件

环路条件

4.4死锁的预防与避免

互斥条件不可禁止

去掉“请求保持条件”

死锁的预防

- 方法
- 采用资源的有序分配
  - 占有某些资源的进程，当它有新的资源请求被拒绝时，该进程资源排队，并释放它所占有的资源，进程请求资源时，必须按顺序提出，从而消除了环路
- 采用预先静态分配方法
  - 如果一个进程在开始时请求所需的全部资源，操作系统要求所有进程一次性地申请其所需的全部资源，只有当全部资源满足时，才一次性地分配给进程，这样就不会发生资源占有和等待资源的现象
- 优点：
  - 方法简单
- 缺点：
  - 资源使用率低，增加新设备类型受到限制。
- 缺点：
  - 该种使用顺序与规定顺序不一致的情况，造成资源浪费
  - 进程延迟运行
  - 必须申请、释放，使进程执行无限延迟，不仅延迟了周转时间，还增加了系统开销，降低了系统吞吐量。
  - 用户有时提不出他要使用的全部资源

死锁的预防措施低效！

去掉“环路“条件

去掉“不剥夺条件”

死锁的避免

避免死锁是通过明智的选择，确保系统永远不会到达死锁点 。即动态地决定是否分配资源给进程！

安全状态—与—不安全状态

- 安全状态是指系统至少存在一个安全序列 $<P_1, P_2, \ldots, P_n>$，按照这个序列为进程分配资源，直到满足最大需求，每个进程都可顺序完成。
- 若系统不存在这样一个安全序列，则系统处于不安全状态。

安全状态

——实例

系统有三个进程P1、P2、P3，共有12台磁带机，进程P1要求10台，P2要求4台，P3要求9台。在T0时刻，进程P1、P2、P3已获得5、2、2台，尚有3台未分配，问：系统是否处于安全状态？

进程	最大需求	已分配	可用
P1	10	5	3
P2	4	2	
P3	9	2	

存在安全序列<P2,P1,P3>

银行家算法

算法描述

- 进行资源预分配
- 实施安全检测
  - 安全：真正资源分配
  - 不安全：回到预分配前状态

数据结构定义

Resource = $(R_1, R_2, \ldots, R_m)$　　系统中每种资源的总量

Available = $(V_1, V_2, \ldots, V_m)$　　没有分配的每种资源总量

$$\text{Need} = \begin{pmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{21} & \cdots & C_{2m} \\ \cdots\cdots & & & \\ \cdots\cdots & & & \\ C_{n1} & C_{n2} & \cdots & C_{nm} \end{pmatrix}$$

每个进程当前对每种资源的需求

$$\text{Allocation} = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{21} & \cdots & A_{2m} \\ & \cdots\cdots & & \\ & \cdots\cdots & & \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{pmatrix}$$

当前分配情况

安全状态

——实例

(a)初始状态：

- **4个进程**
- **3类资源**

(b)

- **让P1运行，不行**
- **让P2运行，OK**

(c)

- **让P1运行，OK**

(d)

- **让P3运行，OK**

存在安全序列

<P2,P1,P3,P4>

Need矩阵

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Allocation矩阵

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Resourse矩阵

R1	R2	R3
9	3	6

Available矩阵

R1	R2	R3
0	1	1

（a）初始状态

Need矩阵

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Allocation矩阵

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Available矩阵

R1	R2	R3
6	2	3

（b）P2 运行完毕

Need矩阵

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Allocation矩阵

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Available矩阵

R1	R2	R3
7	2	3

（c）P1 运行完毕

Need矩阵

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Allocation矩阵

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Available矩阵

R1	R2	R3
9	3	4

（d）P3 运行完毕

不安全状态

——实例

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Need矩阵

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation矩阵

R1	R2	R3
9	3	6

Resourse矩阵

R1	R2	R3
1	1	2

Available矩阵

（a）初始状态

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Need矩阵

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation矩阵

R1	R2	R3
0	1	1

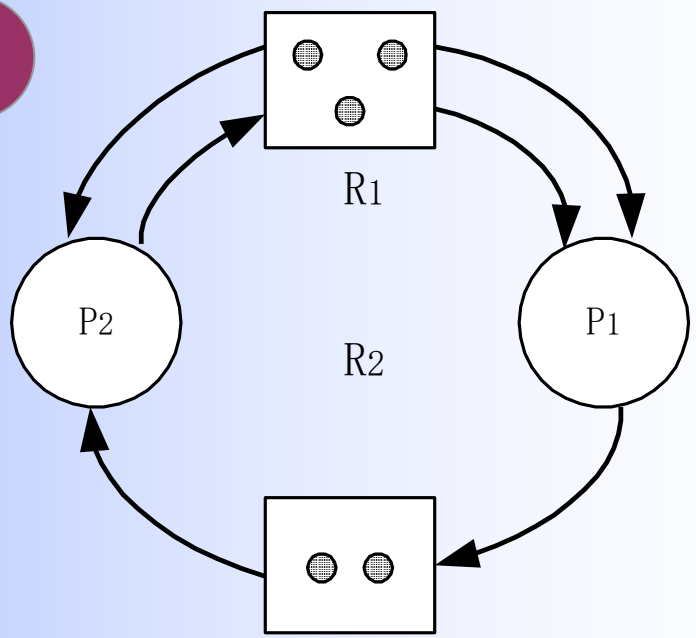
Available矩阵

（b）

- 图（a）定义的矩阵，假设P_2请求1个R_1和1个R_3，如果同意了这个请求，系统的状态回到上图的（a），前面已经分析了这是一个安全状态。
- 但假如在图（a）的状态下P_1请求1个R_1和1个R_3，如果满足P_1的请求，则系统就有了图2-31（b）的状态，
- 状态(b)是不是安全呢？答案是：不安全。

4.5死锁的检测与解除

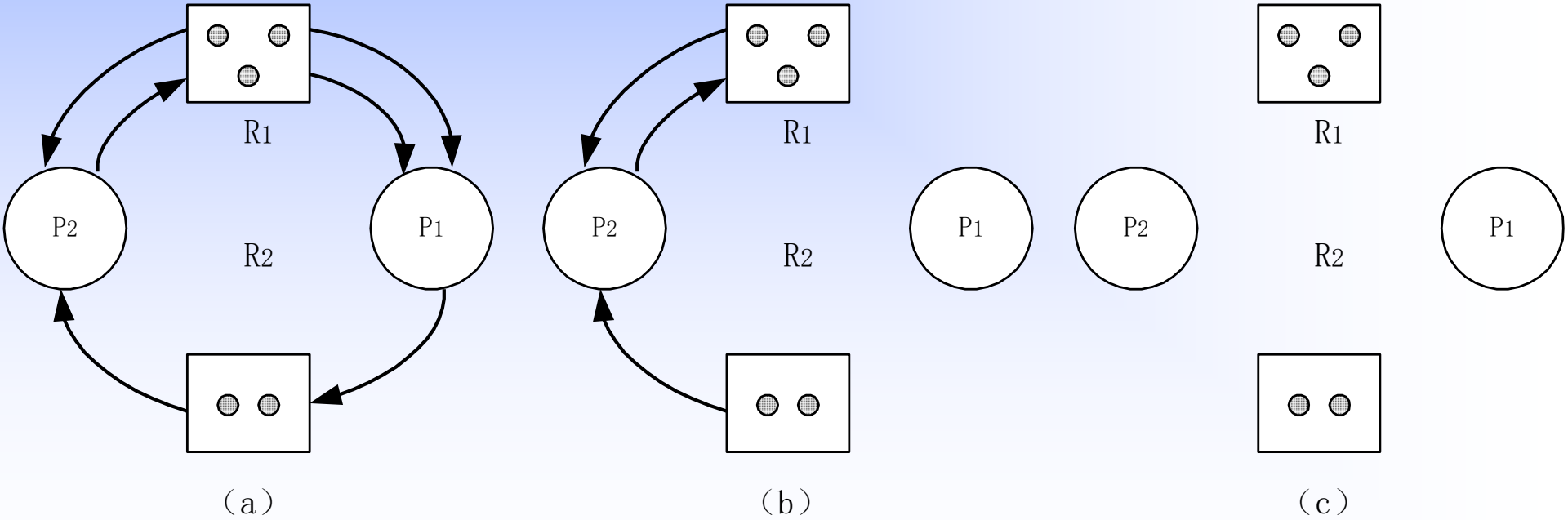
死锁的检测



- 资源分配图
  - 该图是由一组结点N和一组边E组成的一对偶G=（N，E）
  - N被分成两个互斥的子集，一组进程结点P={p1,p2,…,pn}，一组资源结点R={r1,r2,…,rn}，N=PUR
  - 凡属于E中的一个边e∈ E，都连接着P中的一个结点和R中的一个结点，e={pi,rj}是资源请求边，由进程pi指向资源rj，它表示进程pi请求一个单位的资源rj。e={rj，pi}是资源分配边，由资源rj指向进程pi ，它表示一个单位的资源rj分配给进程pi。

资源分配图的简化

- 在图中找出一个既不阻塞又非独立的进程结点pi，消去pi所有的请求边和分配边，使之成为孤立结点。
- 在进行一系列简化后，能消去图中所有的边，使所有进程都成为孤立结点，则称该图是可以完全简化的，否则若不能通过任何过程使该图完全简化，则称该图是不可完全简化的。



当且仅当S状态的资源分配图是不可完全简化的，S为死锁状态。

死锁的解除

- 当发现死锁时，应立即把它们从死锁中解脱出来，常采用的两种方法是：
- 剥夺资源
  - 从其它进程剥夺足够数量的资源给死锁进程。
- 撤消进程：撤消的原则是
  - 为解除死锁状态所需撤消的进程数目最小。
  - 撤消进程所付出的代价最小。

鸵鸟算法

- 对于死锁最简单的方法就是鸵鸟算法：把头埋在沙子里，假装什么也没有发生。每个人对该方法的看法不同。数学家认为这种方法根本不能接受，不管多大代价一定要彻底解决死锁的问题；工程师则要了解发生死锁的概率，系统因各种原因崩溃的概率以及死锁的严重程度。如果死锁每五年发生一次，而系统每周都会因各种硬件或软件故障而崩溃一次，那么大多数工程师则倾向于不要以性能的代价来消除死锁。